

KUBERNETES-07

Task 2: Taint a Node and Schedule a Tolerant Pod Taint a node with special=true:NoSchedule. Create a pod with a toleration that matches the taint, allowing it to be scheduled on the tainted node.

Step 1: Check your nodes

```
ubuntu@k8s-master:~$ kubectl get nodes
NAME           STATUS    ROLES    AGE   VERSION
k8s-master     Ready     control-plane   8d    v1.29.15
worker01       Ready     <none>        8d    v1.29.15
worker02       Ready     <none>        8d    v1.29.15
ubuntu@k8s-master:~$
```

Step 2: Add the taint & check:

```
ubuntu@k8s-master:~$ kubectl taint nodes worker01 special=true:NoSchedule
node/worker01 tainted
ubuntu@k8s-master:~$ kubectl describe node worker01 | grep Taints
Taints:              special=true:NoSchedule
ubuntu@k8s-master:~$
```

What does NoSchedule mean?

It means:

Kubernetes will NOT schedule a new Pod on this node unless the Pod has a matching toleration.

Existing Pods are not automatically removed.

Step 3: Create a Pod with toleration

```
ubuntu@k8s-master:~$ nano tolerant-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: tolerant-pod
spec:
  tolerations:
    - key: "special"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"

  containers:
    - name: nginx
      image: nginx
```

Step 4: Apply & Check:

```
ubuntu@k8s-master:~$ kubectl apply -f tolerant-pod.yaml
pod/tolerant-pod created
```

```
ubuntu@k8s-master:~$ kubectl get pod tolerant-pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP          NODE    NOMINATED NODE   READINESS GATES
tolerant-pod   1/1     Running   0          100s   10.244.1.83 worker01   <none>           <none>
```

The important point is that the Pod can run on worker01 because it has the matching toleration.

Task 3: Use NodeSelector to Schedule a Pod on a Specific Node Label a node with env=dev. Create a pod with a nodeSelector that schedules it only on the node labeled env=dev.

The requirement is:

Label a node env=dev and schedule the Pod only on that node.

Step 1: Label worker01

```
ubuntu@k8s-master:~$ kubectl label nodes worker01 env=dev
node/worker01 labeled
ubuntu@k8s-master:~$ kubectl get nodes --show-labels
NAME          STATUS    ROLES    AGE   VERSION   LABELS
k8s-master    Ready     control-plane   8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,os=linux,node-role.kubernetes.io/control-plane=,node.kubernetes.io/exclude-from-external-load-balancers=
worker01      Ready     <none>        8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=dev,kubernetes.io/arch=amd64,os=linux
worker02      Ready     <none>        8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,os=linux
ubuntu@k8s-master:~$
```

Step 2: Create Pod

```
GNU nano 8.7.1
apiVersion: v1
kind: Pod
metadata:
  name: node-selector-pod
spec:
  nodeSelector:
    env: dev
  containers:
  - name: nginx
    image: nginx
```

Step 3: Apply

```
ubuntu@k8s-master:~$ nano node-selector-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f node-selector-pod.yaml
pod/node-selector-pod created
ubuntu@k8s-master:~$
```

Check:

```
ubuntu@k8s-master:~$ kubectl get pod node-selector-pod -o wide
NAME             READY   STATUS    RESTARTS   AGE   IP          NODE    NOMINATED NODE   READINESS GATES
node-selector-pod 1/1     Running   0          43s   10.244.1.84 worker01   <none>           <none>
```

Because:

worker01

|

└─ env=dev

↓

nodeSelector

↓

Pod

Important difference

nodeSelector is a **hard requirement**.

If there is no node with:

env=dev

the Pod cannot be scheduled.

It will remain:

Pending

Task 4: Use Node Affinity with Soft Scheduling Label a node with env=test. Create a pod with PreferredDuringSchedulingIgnoredDuringExecution node affinity, preferring to schedule it on a node labeled env=test. Remove the label and verify the pod continues to run.

Now we learn **Node Affinity**.

There are two important types:

Preferred = Soft

Required = Hard

Task 4 uses:

PreferredDuringSchedulingIgnoredDuringExecution

Think:

"I prefer this node, but if it is not available, you can use another node."

Step 1: Label worker01 & Check:

```

ubuntu@k8s-master:~$ kubectl label nodes worker01 env=test
node/worker01 labeled
ubuntu@k8s-master:~$ kubectl get nodes --show-labels
NAME          STATUS    ROLES    AGE   VERSION   LABELS
k8s-master    Ready     control-plane   8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=node.kubernetes.io/exclude-from-external-load-balancers=
worker01      Ready     <none>      8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=test,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker01,kubernetes.io/os=linux
worker02      Ready     <none>      8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker02,kubernetes.io/os=linux
ubuntu@k8s-master:~$

```

Step 2: Create Pod

```

GNU nano 8.7.1
apiVersion: v1
kind: Pod
metadata:
  name: preferred-affinity-pod
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          preference:
            matchExpressions:
              - key: env
                operator: In
                values:
                  - test
  containers:
    - name: nginx
      image: nginx

```

The important part is:

preferredDuringSchedulingIgnoredDuringExecution:

This means:

"Prefer a node matching my condition, but don't make it mandatory."

Step 3: Apply

```

ubuntu@k8s-master:~$ nano preferred-affinity-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f preferred-affinity-pod.yaml
pod/preferred-affinity-pod created

```

Check:

```

ubuntu@k8s-master:~$ kubectl get pod preferred-affinity-pod -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP           NODE   NOMINATED NODE   READINESS GATES
preferred-affinity-pod 1/1     Running   0           8s    10.244.1.85  worker01   <none>           <none>
ubuntu@k8s-master:~$

```

Step 4: Remove the label

```

ubuntu@k8s-master:~$ kubectl get nodes --show-labels
NAME          STATUS    ROLES    AGE   VERSION   LABELS
k8s-master    Ready     control-plane   8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=node.kubernetes.io/exclude-from-external-load-balancers=
worker01      Ready     <none>      8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker01,kubernetes.io/os=linux
worker02      Ready     <none>      8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker02,kubernetes.io/os=linux
ubuntu@k8s-master:~$

```

Now check the Pod:

kubectl get pod preferred-affinity-pod -o wide

The Pod should continue running.

Why?

Because:

Preferred



Only a preference during scheduling



Pod already running



Removing label does NOT remove Pod

What does IgnoredDuringExecution mean?

This is very important for interviews.

Suppose:

Pod scheduled on worker01

worker01 has env=test

Later:

env=test is remove

Because we use:

IgnoredDuringExecution

Kubernetes says:

"The Pod is already running. I won't evict it just because the label changed."

Easy meaning

During scheduling → check the rule

After Pod is running → ignore label change

Task 5: Implement Node Affinity with Hard Scheduling Create a pod with `RequiredDuringSchedulingIgnoredDuringExecution` node affinity, ensuring it will only be scheduled on a node labeled `env=prod`. Verify the pod cannot be scheduled if no node has the `env=prod` label.

Step 1: Make sure no node has `env=prod`

```
ubuntu@k8s-master:~$ kubectl get nodes --show-labels
NAME          STATUS    ROLES    AGE   VERSION   LABELS
k8s-master    Ready     control-plane  8d   v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=node.kubernetes.io/exclude-from-external-load-balancers=
worker01      Ready     <none>      8d   v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/os=linux
worker02      Ready     <none>      8d   v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/os=linux
ubuntu@k8s-master:~$
```

Step 2: Create Pod

```
GNU nano 8.7.1
apiVersion: v1
kind: Pod
metadata:
  name: required-affinity-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: env
                operator: In
                values:
                  - prod
  containers:
    - name: nginx
      image: nginx
```

Step 3: Apply

```
ubuntu@k8s-master:~$ nano required-affinity-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f required-affinity-pod.yaml
pod/required-affinity-pod created
ubuntu@k8s-master:~$ kubectl get pod required-affinity-pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP        NODE    NOMINATED NODE   READINESS GATES
required-affinity-pod 0/1     Pending   0          31s   <none>    <none>   <none>           <none>
```

You should see:

```
NAME          READY   STATUS
required-affinity-pod 0/1     Pending
```

Why?

Because there is no node with:

`env=prod`

Step 4: See the reason

```
ubuntu@k8s-master:~$ kubectl describe pod required-affinity-pod
```

At the bottom, look at **Events**.

You will see a scheduling message indicating that no node satisfies the Pod's node affinity requirements.

```
node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type            Reason              Age   From                  Message
  ----            -
  Warning         FailedScheduling    3m8s  default-scheduler    0/3 nodes are available: 1 node(s) had untolera
  e affinity/selector. preemption: 0/3 nodes are available: 3 Preemption is not helpful for scheduling.
ubuntu@k8s-master:~$
```

Step 5: Now label worker02

```
ubuntu@k8s-master:~$ kubectl label nodes worker02 env=prod
node/worker02 labeled
```

```
ubuntu@k8s-master:~$ kubectl get nodes --show-labels
NAME          STATUS    ROLES    AGE   VERSION   LABELS
k8s-master    Ready     control-plane  8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node.kubernetes.io/exclude-from-external-load-balancers=
worker01      Ready     <none>      8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/os=linux
worker02      Ready     <none>      8d    v1.29.15   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=prod,kubernetes.io/os=linux
ubuntu@k8s-master:~$
```

Now:

worker02

|

└─ env=prod

Check the Pod:

```
ubuntu@k8s-master:~$ kubectl get pod required-affinity-pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP          NODE    NOMINATED NODE   READINESS GATES
required-affinity-pod  1/1     Running    0          10m   10.244.2.77 worker02  <none>          <none>
```

Before label:

```
Pod
↓
env=prod required
↓
No matching node
↓
Pending
```

After label:

```
worker02
↓
env=prod
↓
Matches requirement
↓
Pod
↓
Running
```

Task 4 vs Task 5

This is one of the **most important things to remember**.

Task	Affinity	Meaning
Task 4	Preferred...	I prefer this node
Task 5	Required...	I must use this node

Task 6: Taint a Node and Use NoExecute with Toleration Seconds Taint a node with special=true:NoExecute. Create a pod with a tolerationSeconds field (e.g., 60 seconds) and observe it gets evicted after 60 seconds on the tainted node.

Step 1: Remove previous labels if needed

```
ubuntu@k8s-master:~$ kubectl label nodes worker02 env-  
node/worker02 unlabeled  
ubuntu@k8s-master:~$
```

Step 2: Add NoExecute taint

We will taint worker01:

```
ubuntu@k8s-master:~$ kubectl taint nodes worker01 special=true:NoExecute
node/worker01 tainted
ubuntu@k8s-master:~$ kubectl describe node worker01 | grep Taints
Taints:              special=true:NoExecute
ubuntu@k8s-master:~$
```

Step 3: Create Pod with 60-second toleration

```
ubuntu@k8s-master:~$ nano noexecute-pod.yaml
ubuntu@k8s-master:~$
```

```
apiVersion: v1
kind: Pod
metadata:
  name: noexecute-pod
spec:
  nodeSelector:
    kubernetes.io/hostname: worker01

  tolerations:
    - key: "special"
      operator: "Equal"
      value: "true"
      effect: "NoExecute"
      tolerationSeconds: 60

  containers:
    - name: nginx
      image: nginx
```

The important part is:

tolerationSeconds: 60

It means:

This Pod can tolerate the NoExecute taint for 60 seconds.

Step 4: Apply & Check:

```
ubuntu@k8s-master:~$ kubectl apply -f noexecute-pod.yaml
pod/noexecute-pod created
```

```
ubuntu@k8s-master:~$ kubectl get pod noexecute-pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE      NOMINATED NODE   READINESS GATES
noexecute-pod 1/1     Running   0          44s   10.244.1.86  worker01  <none>          <none>
```

Step 5: Wait 60 seconds

Because the node has:

special=true:NoExecute

and the Pod has:

tolerationSeconds: 60

the Pod can remain there for approximately 60 seconds after the taint is applied.

Then Kubernetes evicts it.

Check:

```
ubuntu@k8s-master:~$ kubectl get pod noexecute-pod -o wide
Error from server (NotFound): pods "noexecute-pod" not found
ubuntu@k8s-master:~$
```

Why did it disappear?

Your setup was:

```
worker01
|
└─ special=true:NoExecute
    ↓
    noexecute-pod
    |
    └─ tolerationSeconds: 60
```

The Pod was allowed to stay on `worker01` for about 60 seconds.